

Modelltransformationen

Vorlesung Modellgetriebene Software-Entwicklung Wintersemester 2023/24

Dr.-Ing. Erik Burger | 24. November 2023



Willkommen

Was erwartet Sie heute?

1. Einführung

2. Grundlagen

3. Formalisierung

4. QVT

5. Zusammenfassung

Einführung
○○○

Grundlagen
oooooooooooooooooooo

Formalisierung
oooooooooooo

QVT
oooooooooooooooooooo

Zusammenfassung
○○



- ## Dependability of Software-intensive Systems



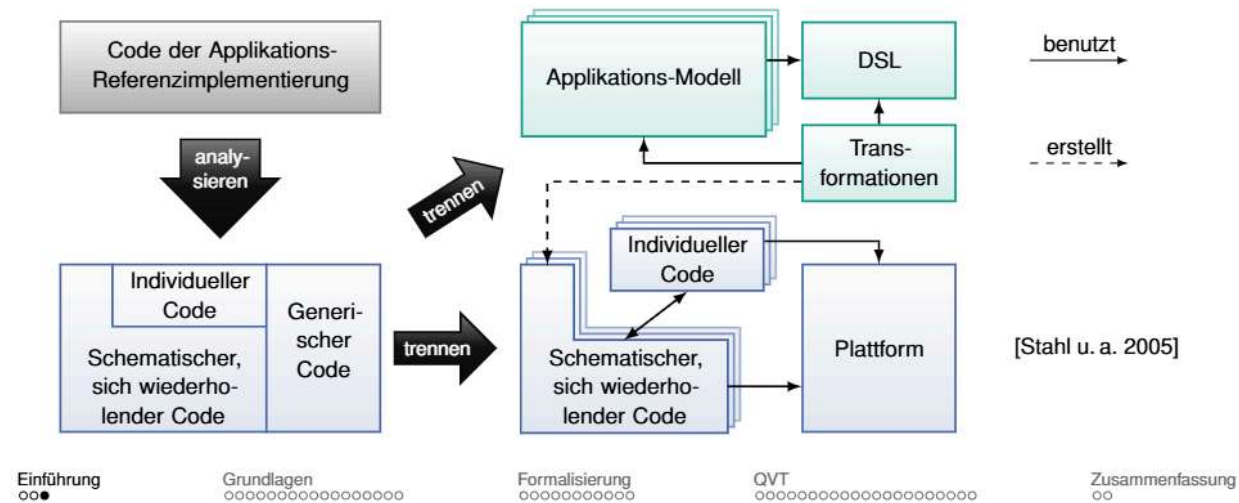
Dependability of Software-intensive Systems

Motivation



Shane Sendall und
Wojtek Kozaczynski. »Model
transformation: the heart and soul of
model-driven software development«. In: *IEEE Software* 20.5 [Sep. 2003],
S. 42–45. ISSN: 0740-7459. DOI:
10.1109/MS.2003.1231150

Wiederholung: MDSD-Prozess



Übersicht

1. Einführung

2. Grundlagen

3. Formalisierung

4. QVT

5. Zusammenfassung

Einführung
○○○

Grundlagen
●○○○○○○○○○○○○○○○○○○

Formalisierung
○○○○○○○○○○○○

QVT
○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○

Zusammenfassung
○○



Modelltransformation

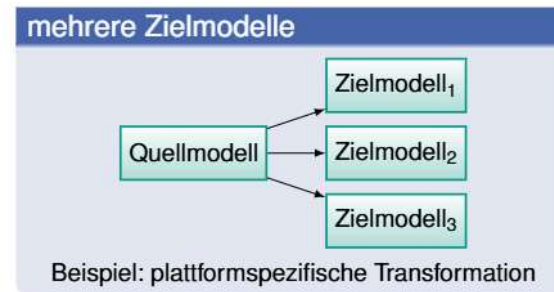
Definition [Kleppe, Warmer und Bast 2003]

A **transformation** is the automatic generation of a target model from a source model, according to a transformation definition.

A **transformation definition** is a set of transformation rules that together describe how a model in the source language can be transformed into a model in the target language.

A **transformation rule** is a description of how one or more constructs in the source language can be transformed into one or more constructs in the target language.

- ## mehrere Quellmodelle
-
- ```
graph LR; Q1[Quellmodell1] --> Z[Zielmodell]; Q2[Quellmodell2] --> Z; Q3[Quellmodell3] --> Z;
```
- Beispiel: Model Merging





Was sind hier im Beispiel Transformation, Transformationsdefinition und Transformationsregeln?

## Wozu verwendet man Transformationen?

### Modelle maschinennutzbar machen

- Kostenersparnis
- Produktivitätsgewinn
- Qualitätssteigerung
- Verringerte Wartungskosten

### Trennung Domäne/Technologie

- angewandt im MDA-Prozess
- bessere Wartbarkeit
- Trennung  
Domänen-Experte/Technologie-Experte

### Wiederverwendung

- der Transformationen selbst oder von Teilen (inter)
- innerhalb einer Transformation z.B. von generiertem Code (intra)

Einführung  
○○○

Grundlagen  
○○○○○●○○○○○○○○○○

Formalisierung  
○○○○○○○○○○○○

QVT  
○○○○○○○○○○○○○○○○○○○○

Zusammenfassung  
○○

## Klassifikation

Was wird transformiert?

### Technischer Raum

- Wird durch das Meta-Metamodell definiert (z.B. XML, MOF)
- Transformationen können innerhalb eines technischen Raums oder zwischen unterschiedlichen technischen Räumen stattfinden

### Umformulierung/Übersetzung

- **endogen:** Transformation zwischen Instanzen desselben Metamodells (Umformulierung).  
*Beispiel:* Optimierung, Refactoring, Vereinfachung/Normalisierung
- **exogen:** Transformation zwischen Instanzen unterschiedlicher Metamodelle (Übersetzung).  
*Beispiel:* Synthese (Code-Generierung, Verfeinerung), Reverse Engineering, Migration

[Mens und Van Gorp 2006]

Einführung  
○○○

Grundlagen  
○○○○○○●○○○○○○○○○○

Formalisierung  
○○○○○○○○○○○○○

QVT  
○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○

Zusammenfassung  
○○

## Klassifikation

### horizontal/vertikal

- **horizontal:** Quell- und Zielmodell sind auf demselben Abstraktionsniveau
- **vertikal:** Quell- und Zielmodell sind auf unterschiedlichem Abstraktionsniveau

### syntaktisch/semantisch

- **syntaktische** Transformation: nur die Syntax der Modelle wird berücksichtigt.  
*Beispiel:* Parser, der Programmcode in Syntaxbaum umwandelt; Formatumwandlung
- **semantische** Transformation: komplexere Transformationen, die die Semantik der Modelle berücksichtigen  
*Beispiel:* Refactoring, Optimierung

[Mens und Van Gorp 2006]

Einführung  
○○○

Grundlagen  
○○○○○○●○○○○○○○○

Formalisierung  
○○○○○○○○○○○○

QVT  
○○○○○○○○○○○○○○○○○○○○

Zusammenfassung  
○○

## Aufgabe: Klassifikation

### Aufgabe: Orthogonale Klassifikation (mit Beispielen)

Bitte fügen Sie die folgenden  
Fälle in die Tabelle ein:

- Code-Generierung
- Formelles Refinement
- Sprachmigration
- Refactoring

|         | horizontal | vertikal |
|---------|------------|----------|
| endogen |            |          |
| exogen  |            |          |

[Mens und Van Gorp 2006]

» Lösung siehe Anhang (Seite 60)

Einführung  
○○○

Grundlagen  
○○○○○○○○●○○○○○○○○

Formalisierung  
○○○○○○○○○○○○

QVT  
○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○

Zusammenfassung  
○○

## Charakteristiken

### Automationsgrad

- Unterscheidung zwischen automatisierbaren Transformationen und (zumindest teilweise) manuell ausgeführten Transformationen
- bei Zweideutigkeiten sind oft manuelle Interventionen notwendig
- **Beispiel:** Transformation zwischen Anforderungs-Modell und Analysemodell

### Komplexität der Transformation

- Beispiel für Transformation mit geringer Komplexität: Modell-Refactorings
- Beispiel für Transformation mit hoher Komplexität: Parser, Compiler, Code-Generator
- komplexe Transformationen benötigen andere Techniken und Werkzeuge als einfache

[Mens und Van Gorp 2006]

Einführung  
○○○

Grundlagen  
○○○○○○○○●○○○○○○○

Formalisierung  
○○○○○○○○○○○

QVT  
○○○○○○○○○○○○○○○○○○○○

Zusammenfassung  
○○



## Charakteristiken



### Erhaltung von Eigenschaften

- Jede Transformation erhält gewisse Eigenschaften der Quellmodelle im Zielmodell
- Beispiel: Refactorings/Restrukturierungen modifizieren die Struktur, aber erhalten das (externe) Verhalten
- Beispiel: Verfeinerungstransformationen

[Mens und Van Gorp 2006]

Einführung  
○○○

Grundlagen  
○○○○○○○○○○●○○○○○○○

Formalisierung  
○○○○○○○○○○○

QVT  
○○○○○○○○○○○○○○○○○○○○○

Zusammenfassung  
○○

## Transformationssprachen

Für die Definition von Modelltransformationen existiert eine Vielzahl von *Modelltransformationssprachen*.

### Anforderungen an Transformationssprachen/-werkzeuge

- *Nutzbarkeit und Nützlichkeit*: Sprachen sollen einen praktischen Zweck erfüllen, aber auch intuitiv und effizient nutzbar sein (abhängig vom Erfahrungsgrad und Gewohnheiten der Entwickler)
- *Verbosity vs. Prägnanz*: so wenige Sprachkonstrukte wie möglich, so viele Spezialkonstrukte wie nötig
- *Performance und Skalierbarkeit*: Ausführung komplexer Transformationen auf großen Modellen sollte möglich sein
- *Erweiterbarkeit*: z.B. durch Plug-Ins
- *Interoperabilität*: Brücken zu anderen technischen Räumen
- *Standardisierung*: Transformationssprachen sollten zu Standards (MOF, UML, XMI) kompatibel sein

Einführung  
○○○

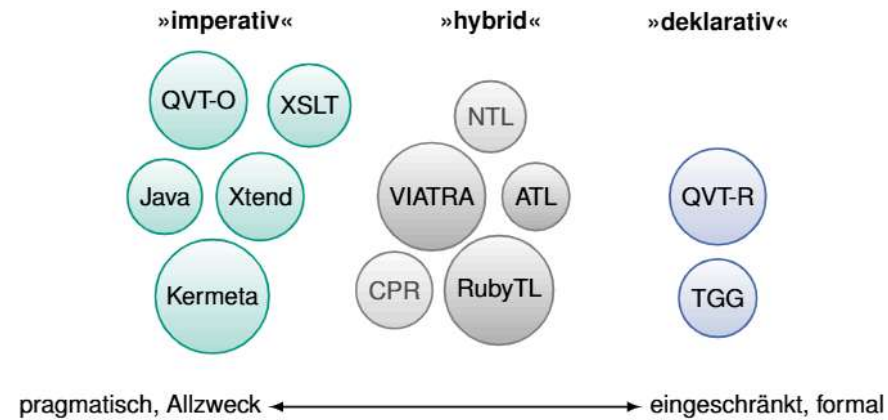
Grundlagen  
○○○○○○○○○○●○○○○○

Formalisierung  
○○○○○○○○○○○

QVT  
○○○○○○○○○○○○○○○○○○○○

Zusammenfassung  
○○

## Welche Sprache für Modelltransformationen?



Einführung  
○○○

Grundlagen  
○○○○○○○○○○●○○○○

Formalisierung  
○○○○○○○○○○○

QVT  
○○○○○○○○○○○○○○○○○○○○

Zusammenfassung  
○○

## Deklarativ vs. Imperativ

Modelltransformationssprachen sind spezielle, domänen-spezifische Programmiersprachen, die bestenfalls auf einen bestimmten technischen Raum beschränkt sind.

### Deklarative Ansätze

- Fokus auf den *Was?*-Aspekt
- Was transformiert werden soll, wird als Relation ausgedrückt

### Imperative Ansätze

- Fokus auf den *Wie?*-Aspekt
- Einzelne Schritte der Transformations-Ausführung werden definiert

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○●○○○

Formalisierung  
○○○○○○○○○○○○○

QVT  
○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○

Zusammenfassung  
○○

- Modell-Traversierung/Navigation
- Trace-Verwaltung
- Bidirektionalität
- implizit ausdrückbar:
  - Navigation des Quellmodells
  - Erstellung des Zielmodells
  - Reihenfolge der Regelausführung

- eine Transformation kann als Funktion gesehen, die ein Quellmodell in ein Zielmodell transformiert
- Falls Funktionen selbst Elemente erster Klasse sind, können sie auch durch die Transformation manipuliert werden
- Nachteil: Modellierung von Zustand schwierig

## Deklarative Sprachen



### Logische Programmierung

Logische Programmiersprachen (Prolog, Mercury, ...) besitzen viele interessante Eigenschaften für Modelltransformationen:

- Backtracking
- Constraint-Propagierung (bei Constraint-Programmiersprachen)
- Unifikation (partiell/vollständig)
- Anfrage-Mechanismus

[Mens und Van Gorp 2006]

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○○○●○

Formalisierung  
○○○○○○○○○○○○

QVT  
○○○○○○○○○○○○○○○○○○○○

Zusammenfassung  
○○

## Imperative Sprachen

- operationale/konstruktive/imperative Ansätze sind notwendig, wenn deklarative Ansätze nicht verwendet werden können bzw. funktionieren
- vor allem dann, wenn die Ausführungsreihenfolge einer Menge von Transformationen explizit angegeben werden muss, sind imperative Ansätze von Vorteil
- Aufruf von externen Programmen (z.B. in Java) möglich
- für Entwickler, die keine Erfahrungen in funktionalem/logischen Programmieren haben, einfacher zu erlernen

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○○○●

Formalisierung  
○○○○○○○○○○○○

QVT  
○○○○○○○○○○○○○○○○○○○○

Zusammenfassung  
○○

# Übersicht

1. Einführung

2. Grundlagen

**3. Formalisierung**

4. QVT

5. Zusammenfassung

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○○○○○○○

**Formalisierung**  
●○○○○○○○○○○

QVT  
○○○○○○○○○○○○○○○○○○○○

Zusammenfassung  
○○



## Formalisierung von Modelltransformationen

- Modelle können als Graphen aufgefasst werden
- Modelltransformationen können also als *Graphtransformationen* formalisiert werden
- Graphtransformation: abstrakter Formalismus, der von der verwendeten Sprache (QVT, ATL, ...) abstrahiert
- visuelle Spezifikation der Transformation
- nutzbar zum Nachweis von Eigenschaften (*Konfluenz*, *Terminierung*)

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○○○○○○○

Formalisierung  
●○○○○○○○○○○

QVT  
○○○○○○○○○○○○○○○○○○○○

Zusammenfassung  
○○

## Wiederholung: Gerichtete Graphen

### Definition

Sei  $C = \langle C_V, C_E \rangle$  ein Paar von *Markierungsalphabeten*. Ein gerichteter, markierter *Graph* über  $C$  ist ein System  $G = (V, E, s, t, l, m)$ . Dabei sind

- $V, E$  endliche Mengen von *Knoten (vertices)* und *Kanten (edges)*
- $s, t : E \rightarrow V$  Abbildungen, die jeder Kante eine *Quelle (source)* und ein *Ziel (target)* zuweisen
- $l : V \rightarrow C_V$  und  $m : E \rightarrow C_E$  Abbildungen, die jedem Knoten eine *Knotenmarkierung* und jeder Kante eine *Kantenmarkierung* zuweisen

[Habel 2015]

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○○○○○○○

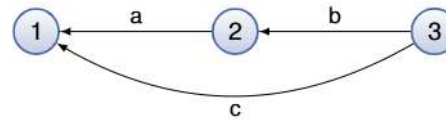
Formalisierung  
○○●○○○○○○○○

QVT  
○○○○○○○○○○○○○○○○○○○○

Zusammenfassung  
○○

## Beispiel: Gerichteter Graph

$V = \{1, 2, 3\}$   
 $E = \{a, b, c\}$   
 $s(a) = 2$   
 $t(a) = 1$   
 $s(b) = 3$   
 $t(b) = 2$   
 $s(c) = 3$   
 $t(c) = 1$



## Graph-Grammatik

### Graphersetzungssystem

Ein **Graphersetzungssystem** ist ein System  $\mathcal{G} = \langle C, \mathcal{R} \rangle$ , wobei  $C$  ein Paar von Markierungsalphabeten und  $\mathcal{R}$  eine Menge von Graphersetzungsregeln über  $C$  ist.

### Graphgrammatik und erzeugte Sprache

Eine **Graphgrammatik** ist ein System  $\mathcal{G} = \langle C, \mathcal{R}, N, S \rangle$ , wobei  $\langle C, \mathcal{R} \rangle$  ein Graphersetzungssystem,  $N = \langle N_V, N_E \rangle$  ein Paar von nichtterminalen Markierungsalphabeten mit  $N_V \subseteq C_V$  und  $N_E \subseteq C_E$  und  $S$  ein Graph über  $C$  ist. Die von  $\mathcal{G}$  erzeugte **Graphsprache**  $L(\mathcal{G})$  besteht aus allen Graphen  $G$ , die mit Hilfe der Regeln in  $\mathcal{R}$  aus  $S$  ableitbar sind ( $\Rightarrow_{\mathcal{R}}^*$ ) und in denen keine nichtterminal markierten Knoten oder Kanten vorkommen:

$$L(\mathcal{G}) = \{ G \in \mathcal{G}_{C \setminus N} \mid S \Rightarrow_{\mathcal{R}}^* G \}$$

Dabei bezeichnet  $C \setminus N$  das Paar  $\langle C_V \setminus N_V, C_E \setminus N_E \rangle$ .

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○○○○○○○

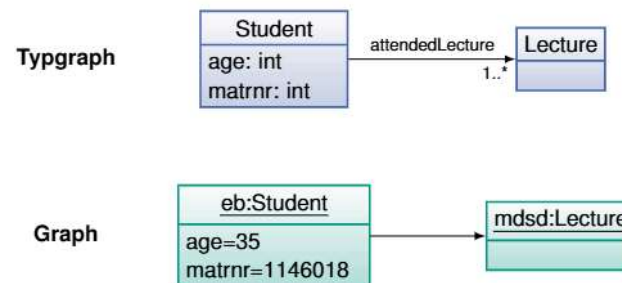
Formalisierung  
○○○○●○○○○○

QVT  
○○○○○○○○○○○○○○○○○○○○

Zusammenfassung  
○○

## Darstellung von Modellen und Metamodellen als Graphen

- Jedes Metamodell kann als attribuiertes Typgraph dargestellt werden
- Jedes Modell kann als ein attribuiertes Instanzengraph über den attribuierten Typgraphen gesehen werden



## Graphersetzungsregel



### Definition

Eine **Graphersetzungsregel** über  $C$  hat die Form  $r = (L, K, R)$ , wobei  $L$ ,  $K$  und  $R$  Graphen über  $C$  sind. Es gilt  $L \supseteq K \subseteq R$ .

$L$  heißt **linke Seite**,  $R$  **rechte Seite** und  $K$  der **Klebegraph** von  $r$ .

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○○○○○○○

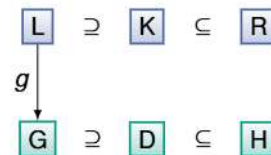
Formalisierung  
○○○○○○●○○○○

QVT  
○○○○○○○○○○○○○○○○○○○○

Zusammenfassung  
○○

## Graphersetzungsregel

Anwendung von  $r = (L, K, R)$  mit  $(L \supseteq K \subseteq R)$  auf  $G$ :



- 1 Wähle ein Vorkommen von  $L$  in  $G$ , d.h. einen Graphmorphismus  $g : L \rightarrow G$ .
- 2 Überprüfe die Kontakt- und Identifikationsbedingung (siehe [Habel 2015])
- 3 Lösche  $g(L \setminus K)$ , d.h. alle Kanten in  $g_E(E_L \setminus E_K)$  und alle Knoten in  $g_V(V_L \setminus V_K)$ . Zwischenergebnis:  
 $D = G \setminus g(L \setminus K)$ .
- 4 Füge  $R \setminus K$  hinzu, d.h. alle Knoten in  $V_R \setminus V_K$  und alle Kanten in  $E_R \setminus E_K$ . Ergebnis:  $H = D \cup (R \setminus K)$ .

[Habel 2015]

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○○○○○○○

Formalisierung  
○○○○○○●○○○

QVT  
○○○○○○○○○○○○○○○○○○○○

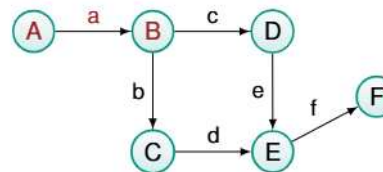
Zusammenfassung  
○○

## Beispiel

### Graphersetzungsgregel



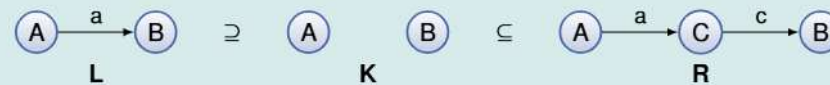
Anwendung:



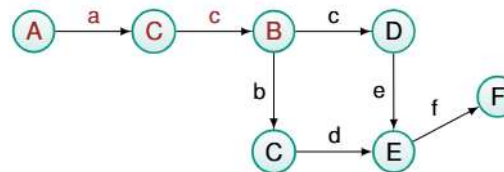


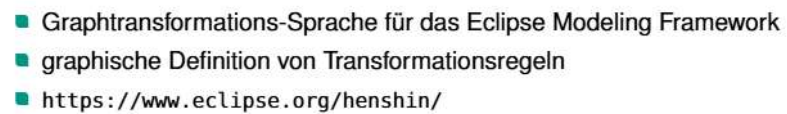
## Beispiel

### Graphersetzungsgregel



Anwendung:





## Graphtransformationen: Zusammenfassung



- Graphtransformationen können auf Modelle und Metamodelle angewendet werden
- Graphentheorie ist theoretische Grundlage für Modelltransformationen
- Eigener Forschungszeit: *International Conference of Graph Transformations (ICGT)*

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○○○○○○○

Formalisierung  
○○○○○○○○○○●

QVT  
○○○○○○○○○○○○○○○○○○○○

Zusammenfassung  
○○

# Übersicht

1. Einführung

2. Grundlagen

3. Formalisierung

4. QVT

5. Zusammenfassung

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○○○○○○○

Formalisierung  
○○○○○○○○○○○○

QVT  
●○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○

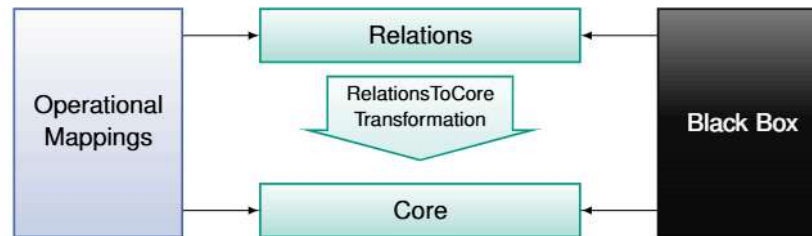
Zusammenfassung  
○○



- ## Anwendungsszenarien

- ## Dependability of Software-intensive Systems

## Architektur



- Relations: Musterabgleich und Template-Erstellung, Traces zwischen Modellelementen
- Core: minimale Erweiterungen von EMOF und OCL; Trace-Klassen sind MOF-Modelle

[Meta Object Facility (MOF) 2.0 Query/View/Transformation Specification 2015]

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○○○○○○○

Formalisierung  
○○○○○○○○○○○○○

QVT  
○○●○○○○○○○○○○○○○○○○○○○○

Zusammenfassung  
○○

## QVT-R

### QVT Relational

- Transformationen in QVT-R werden als Relationen ausgedrückt
- zur Erinnerung: Eine Relation über Mengen  $A$  und  $B$  ist eine Untermenge des kartesischen Produkts dieser Mengen:  $R \subseteq A \times B$
- eine Relation ist nicht notwendigerweise eine Funktion (nur wenn *linkstotal* und *rechtseindeutig*)

### Engines

- ModelMorf (seit 2007 nicht mehr weiterentwickelt)
- MediniQVT (seit 2011 nicht mehr weiterentwickelt)
- Eclipse MTT/QVT Declarative<sup>a</sup> (Version 0.28.1)

<sup>a</sup>März 2022, <http://www.eclipse.org/mmt/downloads/?project=qvtd>

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○○○○○○○

Formalisierung  
○○○○○○○○○○○○○

QVT  
○○○●○○○○○○○○○○○○○○○○○○

Zusammenfassung  
○○

## Bidirektionalität

- QVT-R unterstützt bidirektionale Transformationen
- bidirektional heißt nicht notwendigerweise bijektiv
- die in QVT-R beschriebene Relation legt fest, welche Bedingungen zwischen Modellen gelten sollen
- Bedingungen werden überprüft (check) und ggf. erzwungen (enforce)

[Stevens 2010]

### Ausführung

Welches Model Quell-, welches Zielmodell ist, wird erst bei *Ausführung* der Transformation festgelegt, nicht in der Transformation selbst

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○○○○○○○

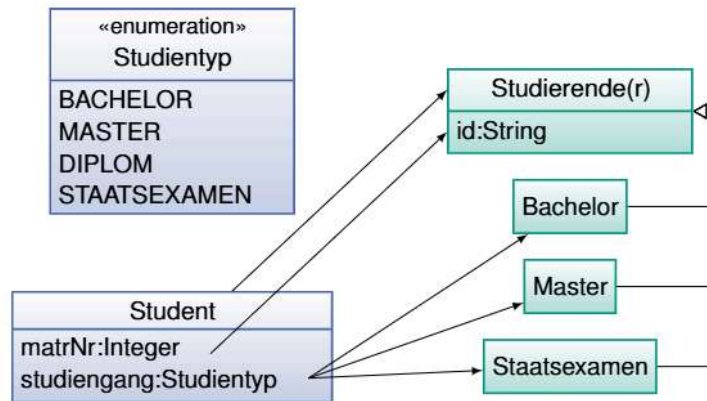
Formalisierung  
○○○○○○○○○○○○

QVT  
○○○○●○○○○○○○○○○○○○○○○○○

Zusammenfassung  
○○



## Beispiel


 Einführung  
 ○○○

 Grundlagen  
 ○○○○○○○○○○○○○○○○○○○○○

 Formalisierung  
 ○○○○○○○○○○○○

 QVT  
 ○○○○○●○○○○○○○○○○○○○○○○○○○○

 Zusammenfassung  
 ○○

## Beispiel: QVT-R

```

import dipl : 'diplom.ecore'::DIPLOM;
import bolo : 'bologna.ecore'::BOLOGNA;

transformation diplToBologna(diplom:dipl, bologna:bolo) {
 top relation StudentToBachelor {
 unique_id:String;
 checkonly domain diplom
 student:Student {matrNr=unique_id};
 enforce domain bologna
 bachelor:Bachelor {id=unique_id};
 when {
 student.studiengang='BACHELOR';
 }
 }
}

```

 Einführung  
 ○○○

 Grundlagen  
 ○○○○○○○○○○○○○○○○○○○

 Formalisierung  
 ○○○○○○○○○○○○

 QVT  
 ○○○○○●○○○○○○○○○○○○○○○○○○

 Zusammenfassung  
 ○○

## QVT-R: Domänenelemente

### Domain

- **domain**-Elemente sind der Startpunkt für eine (Graph-)Mustersuche
- alle anderen Elemente müssen über OCL-Navigation vom **domain**-Element aus erreichbar sein
- pro Transformation können beliebige Domänen definiert werden

### Checkonly

- Domänenelemente mit **checkonly** werden nur lesend überprüft
- Verletzungen der Bedingung werden protokolliert

### Enforce

- Elemente mit **enforce** können erstellt, modifiziert oder gelöscht werden
- jedoch nur, wenn sie zum *Zielmodell* gehören

 Einführung  
 ○○○

 Grundlagen  
 ○○○○○○○○○○○○○○○○○○○○○

 Formalisierung  
 ○○○○○○○○○○○○

 QVT  
 ○○○○○○○●○○○○○○○○○○○○○○○○○○○○

 Zusammenfassung  
 ○○

## QVT-R: When und Where

### When

- bindet Variablen, bevor die Domänenmuster ausgewertet werden
- wird ignoriert, wenn keine Variablen gebunden werden können
- ähnlich einer Vorbedingung

### Where

- muss immer nach **true** auswerten
- kann benutzt werden, um Unter-Relationen aufzurufen

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○○○○○○○

Formalisierung  
○○○○○○○○○○○○

QVT  
○○○○○○○○●○○○○○○○○○○○○○○

Zusammenfassung  
○○

## QVT-O

### QVT Operational

- imperative Variante von QVT
- gute Unterstützung im Eclipse Modelling Framework (auch Debugger)

### Bestandteile einer QVT-O-Transformation

- Kopf
- Main-Methode
- Mapping-Regeln
- Queries (optional)

(ähnlich wie in Java)

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○○○○○○○

Formalisierung  
○○○○○○○○○○○○

QVT  
○○○○○○○○○○●○○○○○○○○○○○○

Zusammenfassung  
○○

## Beispiel: QVT-O

```

modeltype DIPL uses 'http://sdq.ipd.kit.edu/diplom';
modeltype BOLO uses 'http://sdq.ipd.kit.edu/bologna';
transformation diplToBologna(in dp:DIPL, out bl:BOLO);
main() {
 var bachelors = dp->objectsOfType(DIPL::Student)->
 select(s|s.studiengang->exists(e|e=Studenttyp::BACHELOR));
 bachelors->forEach(b) { b->map studToBachelor(); }
}

mapping DIPL::Student::studToBachelor() : BOLO::Bachelor
when {self.matrNr>0} {
 id := self.matrNr.toString();
}

```

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○○○○○○○

Formalisierung  
○○○○○○○○○○○○○

QVT  
○○○○○○○○○○●○○○○○○○○○○

Zusammenfassung  
○○

## ImperativeOCL

QVT-O verwendet *ImperativeOCL* (= OCL mit Nebenwirkungen, turingmächtig)

### Sprachkonstrukte

- Funktionsaufrufe: `map`
- Variablen und Zuweisungen: `var`, `:=`
- Kontrollfluss: `switch/case`, `while/break`, `for/continue`, `if/else`, `return`, ...
- explizite Instanziierung: `object`, `new`
- Exceptions: `assert`, `try`, `raise`, `catch`

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○○○○○○○

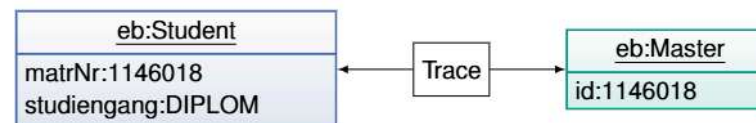
Formalisierung  
○○○○○○○○○○○○

QVT  
○○○○○○○○○○○○●○○○○○○○○○○

Zusammenfassung  
○○

## QVT-O: Trace-Modell

Die Beziehungen der Quellmodelle zu den Zielmodellen werden in einem *Trace-Modell* gespeichert.



Die Informationen Trace-Modell können über die *resolve*-Funktion von QVT-O in Transformationen genutzt werden.

Anwendungsfälle:

- Attribute in bereits erstellten Objekten aktualisieren
- Überprüfung, ob ein Element schon transformiert wurde



## QVT: Praktisch



### SDQ-Wiki

Links zu Tutorials, Tips & Tricks usw.:

<https://sdq.kastel.kit.edu/wiki/QVT>

Einführung  
○○○

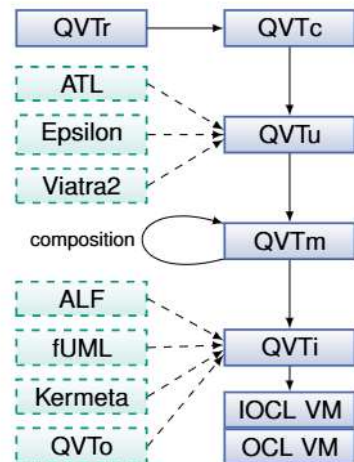
Grundlagen  
○○○○○○○○○○○○○○○○○○○○

Formalisierung  
○○○○○○○○○○○○

QVT  
○○○○○○○○○○○○○○○○●○○○○○○○

Zusammenfassung  
○○

## QVT[a-z]? (Eclipse MMT)



**QVTr:** multidirektionale deklarative Hochsprache

**QVTc:** »einfache« multidirektionale deklarative Sprache

**QVTu:** »einfache« unidirektionale deklarative Sprache

**QVTm:** »minimale« unidirektionale deklarative Sprache

**QVTi:** »sehr kleine« unidirektionale imperative Sprache

**QVTo:** multidirektionale imperative Hochsprache

[http://wiki.eclipse.org/MMT/QVT\\_Declarative\\_Languages](http://wiki.eclipse.org/MMT/QVT_Declarative_Languages)

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○○○○○○○

Formalisierung  
○○○○○○○○○○○○○

QVT  
○○○○○○○○○○○○○○○○○○●○○○○○○○

Zusammenfassung  
○○

## ATL



### Atlas Transformation Language

- entwickelt von OBEO und INRIA als Antwort auf den QVT Request For Proposals (RFP)
- wird von OBEO und AtlanMod (früher ATLAS Group) weiterentwickelt
- steht unter *Eclipse Public License*
- Teil des *Eclipse Modeling Projects*
- Teil der AMMA (ATLAS Model Management Architecture)

### Eigenschaften

- hybride Sprache (deklarativ/imperativ)
- verwendet OCL 2.0 zur Navigation
- Transformations-»Zoo« mit über 100 Beispielen: <http://www.eclipse.org/atl/atlTransformations/>

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○○○○○○○

Formalisierung  
○○○○○○○○○○○○○

QVT  
○○○○○○○○○○○○○○○○○○●○○○○○

Zusammenfassung  
○○

## ATL: Eigenschaften

### Unterscheidung Quell- und Zielmodell

- Das Quellmodell kann nur gelesen werden
- Das Zielmodell kann nur geschrieben werden

### Hybride Sprache

- Deklarativer Teil: Regeln mit Matching und automatischer Traceability
- Imperativer Teil: Aufgerufene Regeln und Aktions-Blöcke
- Navigation und Abfragen: OCL 2.0
- Bevorzugter Programmierstil: *deklarativ*

 Einführung  
 ○○○

 Grundlagen  
 ○○○○○○○○○○○○○○○○○○○○○

 Formalisierung  
 ○○○○○○○○○○○○

 QVT  
 ○○○○○○○○○○○○○○○○○○○●○○○○○

 Zusammenfassung  
 ○○

## Beispiel: ATL

```

module diplToBologna;
-- @path Diplom=/student.ecore/model/diplom.ecore
-- @path Bologna=/student.ecore/model/bologna.ecore
create OUT: Bologna from IN: Diplom;

helper context Diplom!Student
 def: isBachelor(): Boolean = self.studiengang=#BACHELOR;

rule BachelorToDiplom {
 from s: Diplom!Student (s.isBachelor())
 to t: Bologna!Bachelor (id <- s.matrNr->toString())
}

```

 Einführung  
 ○○○

 Grundlagen  
 ○○○○○○○○○○○○○○○○○○○○○

 Formalisierung  
 ○○○○○○○○○○○○

 QVT  
 ○○○○○○○○○○○○○○○○○○○●○○○

 Zusammenfassung  
 ○○

## Ausführungsreihenfolge

- die Reihenfolge des Matchings und Ausführen der deklarativen Regeln ist nicht spezifiziert
- die Reihenfolge, in der Elemente gebunden (erstellt) werden, ist nicht spezifiziert
- dennoch deterministisch, da Quellmodelle nicht geändert und Zielmodelle nicht navigiert werden können

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○○○○○○○

Formalisierung  
○○○○○○○○○○○○

QVT  
○○○○○○○○○○○○○○○○○○○○●○○

Zusammenfassung  
○○

## KM3

- ATL besitzt ein eigenes Meta-Metamodell: KM3 (Kernel Meta-Metamodel)
- jedoch in Eclipse auch mit Ecore-basierten (d.h. MOF-kompatiblen) Metamodellen benutzbar

### Konkrete textuelle Syntax von KM3 (Beispiel)

```
package Diplom {
 abstract class Person {
 attribute name : String;
 }

 class Student extends Person {
 attribute matrNr: Integer;
 attribute studiengang: Studenttyp;
 }
}
```

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○○○○○○○

Formalisierung  
○○○○○○○○○○○○○○

QVT  
○○○○○○○○○○○○○○○○○○○○●○○

Zusammenfassung  
○○

## Zusammenfassung: QVT

- QVT ist der MOF-Standard für die Definition von Modelltransformationen
- deklarativer (QVT-R) und imperativer (QVT-O) Teil, die sich (theoretisch) auf einen gemeinsamen Kern beziehen
- unterschiedliche Implementierungen unterschiedlicher Güte
- stabilste Engines: Eclipse QVT-O, ATL
- Weiterentwicklung im Eclipse-MTT-Projekt fragwürdig

|                   |                                    |                                |                              |                       |
|-------------------|------------------------------------|--------------------------------|------------------------------|-----------------------|
| Einführung<br>○○○ | Grundlagen<br>○○○○○○○○○○○○○○○○○○○○ | Formalisierung<br>○○○○○○○○○○○○ | QVT<br>○○○○○○○○○○○○○○○○○○○○● | Zusammenfassung<br>○○ |
|-------------------|------------------------------------|--------------------------------|------------------------------|-----------------------|



# Übersicht

1. Einführung

2. Grundlagen

3. Formalisierung

4. QVT

**5. Zusammenfassung**

Einführung  
ooo

Grundlagen  
oooooooooooooooooooo

Formalisierung  
oooooooooooo

QVT  
oooooooooooooooooooo

**Zusammenfassung**  
●o

## Was haben Sie heute gelernt?

- Modelltransformationen werden durch spezielle Modelltransformationssprachen und -engines realisiert
- Es existiert eine Vielzahl von Transformationssprachen
- Hauptunterscheidungsmerkmal: Paradigmen *deklarativ* und *imperativ*
- Welche Transformationssprache für welchen Zweck hängt von mehreren Faktoren ab (Eignung des Paradigmas, Qualität der Tools)

### Nächste Sitzung

**Freitag, 1. Dezember, 9:45–11:15 Uhr**

Gebäude 50.34, Raum –102

Einführung  
○○○

Grundlagen  
○○○○○○○○○○○○○○○○○○○○

Formalisierung  
○○○○○○○○○○○○

QVT  
○○○○○○○○○○○○○○○○○○○○

Zusammenfassung  
●

## Literatur I

- [1] Annegret Habel. *Graph Replacement Systems*. Skript der Vorlesung. 2015.
- [2] Anneke G. Kleppe, Jos Warmer und Wim Bast. *MDA Explained: The Model Driven Architecture: Practice and Promise*. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc., 2003. ISBN: 032119442X.
- [3] Tom Mens und Pieter Van Gorp. »A taxonomy of model transformation«. In: *Electronic Notes in Theoretical Computer Science* 152 (2006), S. 125–142.
- [4] *Meta Object Facility (MOF) 2.0 Query/View/Transformation Specification*. Object Management Group. February 2015. URL: <http://www.omg.org/spec/QVT/1.2/>.
- [5] *Meta Object Facility (MOF) 2.0 Query/View/Transformation Specification*. Object Management Group. February 2016. URL: <http://www.omg.org/spec/QVT/1.3/>.
- [6] Shane Sendall und Wojtek Kozaczynski. »Model transformation: the heart and soul of model-driven software development«. In: *IEEE Software* 20.5 (Sep. 2003), S. 42–45. ISSN: 0740-7459. DOI: 10.1109/MS.2003.1231150.

Literatur  
o

## Literatur II



- [7] Thomas Stahl u. a. *Modellgetriebene Softwareentwicklung : Techniken, Engineering, Management*. 1. Aufl. Heidelberg: dpunkt-Verl., 2005. ISBN: 3-89864-310-7.
- [8] Perdita Stevens. »Bidirectional model transformations in QVT: semantic issues and open questions«. In: *Software & Systems Modeling* 9.1 (2010), S. 7–20. ISSN: 1619-1366. DOI: 10.1007/s10270-008-0109-9. URL: <http://dx.doi.org/10.1007/s10270-008-0109-9>.

## Lösung: Klassifikation

### Aufgabe: Orthogonale Klassifikation (mit Beispielen)

Bitte fügen Sie die folgenden  
Fälle in die Tabelle ein:

- Code-Generierung
- Formelles Refinement
- Sprachmigration
- Refactoring

|         | horizontal      | vertikal             |
|---------|-----------------|----------------------|
| endogen | Refactoring     | Formelles Refinement |
| exogen  | Sprachmigration | Code-Generierung     |

[Mens und Van Gorp 2006]

← Aufgabe (Seite 15)

Literatur  
•